

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

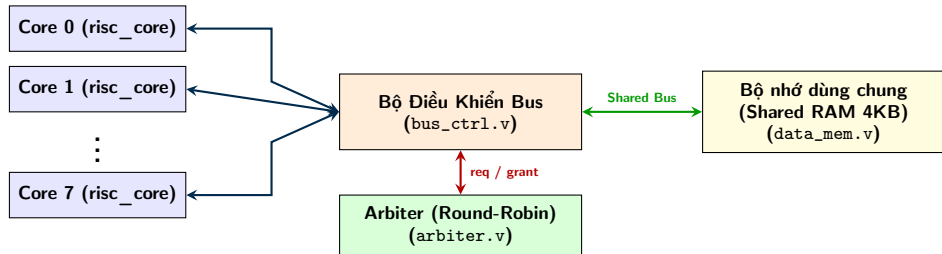
Ngày 18 tháng 6 năm 2026

1 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

- 1 Thiết Kế Hệ Thống (Design)
 - Cấu trúc tổng thể
 - Kiến trúc Pipeline và Logic Điều khiển

Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

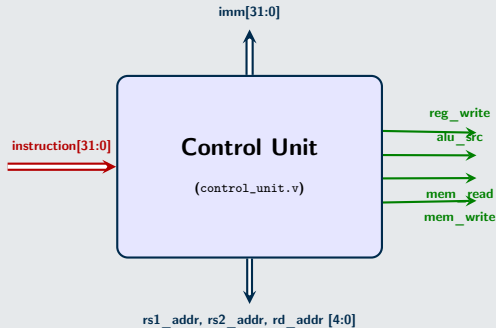
1 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

Control Unit

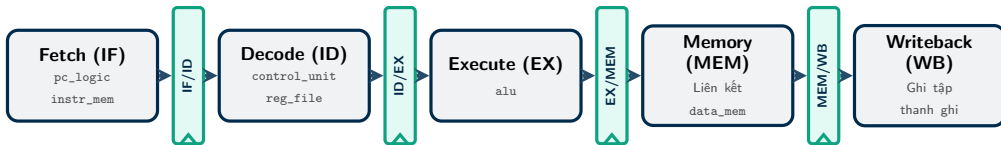
- **Nhiệm vụ:** Sử dụng Combinational Logic để giải mã các trường Opcode, Funct3, Funct7 của Instruction và lập tức sinh các tín hiệu điều khiển cho hệ thống Datapath.

Sơ đồ Input Output Control Unit



Phân bố phần cứng trên 5 tầng Pipeline

Hệ thống **Pipeline Registers** đóng vai trò là các vách ngăn đồng bộ, cô lập dữ liệu giữa các giai đoạn xử lý:



Nguyên lý điều khiển luồng trên các Thanh ghi đệm Pipeline

- **Clock Synchronization:** Dữ liệu/lệnh từ tầng trước chỉ được phép dịch chuyển sang tầng sau chính xác tại sườn lên của xung Clock (posedge clk).
- **Pipeline register:** Giữ cố định trạng thái xử lý độc lập của 5 lệnh khác nhau cùng lúc, ngăn chặn hiện tượng lẫn át hay trộn lẫn dữ liệu giữa các tầng Datapath.
- **Pipeline Interlock:** Là nơi trực tiếp tiếp nhận các lệnh (hold để tạo Stall, hoặc clear để tạo Flush/Bubble) từ bộ điều khiển trung tâm.

PIPELINE REGISTERS: Fetch / Decode (IF/ID Register)

- **Vai trò:** Chốt giữ Instruction 32-bit vừa đọc ra từ Instruction Memory và địa chỉ PC tương ứng để chuyển giao sang giai đoạn Decode.
- **Cơ chế Hazard:** Nhận tín hiệu hold khi kích hoạt Load-Use Stall hoặc Memory Stall để đóng băng trạng thái; nhận tín hiệu clear để thực hiện Flush lệnh rác khi rẽ nhánh Taken sai.

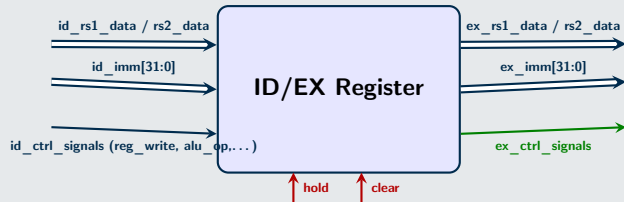
Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_if_id.v



PIPELINE REGISTERS Decode / Execute (ID/EX Register)

- **Vai trò:** Tiếp nhận và chuyển tiếp toàn bộ các tín hiệu điều khiển từ Control Unit, giá trị các toán hạng đọc từ Register File (`rs1_data`, `rs2_data`) và số Immediate đã mở rộng sang giai đoạn Execute cho ALU.
- **Cơ chế Hazard:** Nhận tín hiệu `clear = 1` để chủ động chèn một Bubble (NOP) vào tầng Execute khi xảy ra xung đột Load-Use.

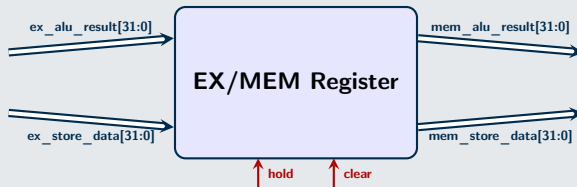
Sơ đồ cổng giao tiếp (Black Box) - `pipe_reg_id_ex.v`



PIPELINE REGISTERS Execute / Memory (EX/MEM Register)

- **Vai trò:** Chốt giữ kết quả tính toán từ đầu ra của ALU, dữ liệu cần ghi vào Data Memory (rs2_data) của lệnh Store, và địa chỉ thanh ghi đích rd.
- **Cơ chế Hazard:** Nhận tín hiệu hold từ mạch Memory Stall (mem_wait) nhằm giữ nguyên thông tin khi Shared Bus đang bận phân xử quyền truy cập bộ nhớ đa nhân.

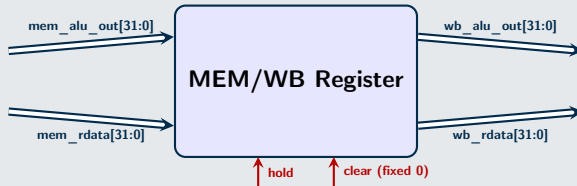
Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_ex_mem.v



PIPELINE REGISTERS Memory / Writeback (MEM/WB Register)

- **Vai trò:** Chốt giữ dữ liệu đọc lên từ Data Memory (rdata) hoặc kết quả trực tiếp từ ALU để sẵn sàng đưa về cổng ghi của Register File.
- **Cơ chế Hazard (Đã vá lỗi):** Loại bỏ hoàn toàn chân clear độc hại, chuyển sang sử dụng chân hold đồng bộ. Khi gặp Memory Stall, dữ liệu tầng này đứng im để bảo toàn dữ liệu, tránh lỗi mất thông tin trên Pipeline.

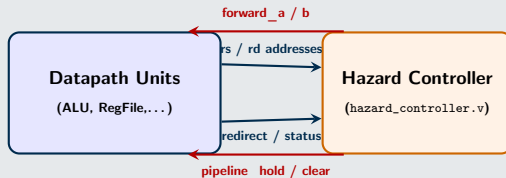
Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_mem_wb.v



Nguyên tắc đóng gói:

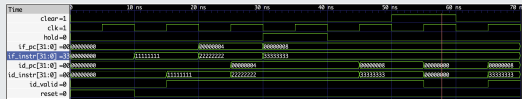
- Giữ nguyên 100% giao tiếp Ports ra hệ thống Shared Bus ngoại vi và mô hình 8-Core.
- Thực hiện instantiate 6 module con độc lập bên trong ruột và kết nối đồng bộ bằng mạng lưới wire.

Sơ đồ kết nối khối điều khiển trung tâm trong Core



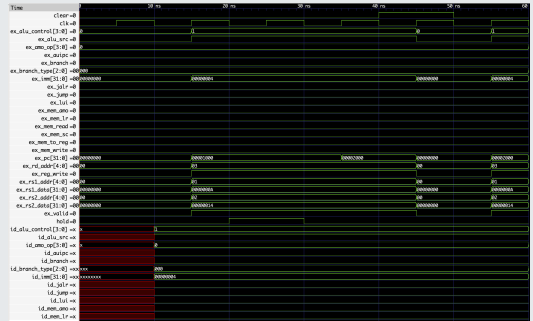
Pipeline Registers Unit Testing - IF/ID và ID/EX

1. Kết quả Simulation pipe_reg_if_id



Phân tích Waveform: Xác thực phản hồi chính xác khi tín hiệu hold dựng lên, giá trị PC và Instruction được khóa cứng, không bị ảnh hưởng bởi sườn Clock.

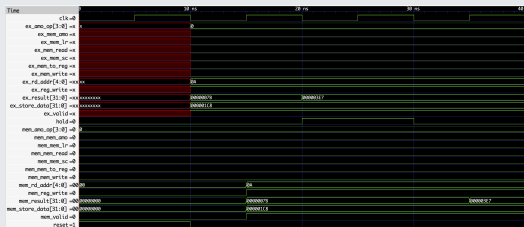
2. Kết quả Simulation pipe_reg_id_ex



Phân tích Waveform: Tín hiệu clear (Flush) kích hoạt thành công, xóa sạch các cờ điều khiển về trạng thái 0 (NOP Bubble) ngay chu kỳ kế tiếp.

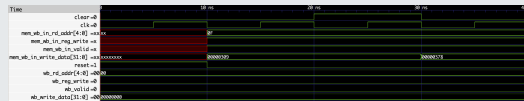
Pipeline Registers Unit Testing - EX/MEM và MEM/WB

3. Kết quả Simulation pipe_reg_ex_mem



Phân tích Waveform: Các toán hạng từ ALU và dữ liệu Store được chốt giữ và chuyển giao đồng bộ sang Data Memory theo sườn lên của Clock.

4. Kết quả Simulation pipe_reg_mem_wb



Phân tích Waveform: Cơ chế vá lỗi thực tế; sử dụng chân hold thay vì clear. Khi xảy ra Memory Stall (mem_wait), dữ liệu tầng này đứng im để bảo toàn tính toàn vẹn.

Hazard Detection & Forwarding

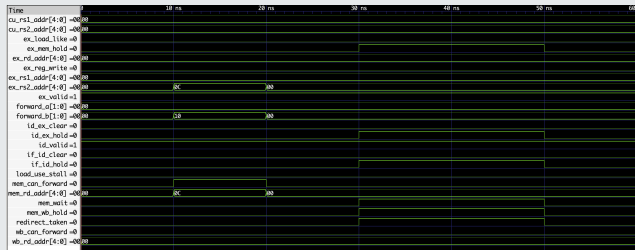
- **Tư duy tích hợp:** Tập trung hóa (Centralized Control) toàn bộ logic xử lý Load-Use Stall, đi tắt dữ liệu (Forwarding) và quản lý trạng thái bận hệ thống (mem_wait) vào một bộ điều khiển duy nhất nhằm tối ưu tài nguyên phần cứng.

Sơ đồ khối điều khiển trung tâm hệ thống - Hazard Controller



Hazard Controller Unit Testing

Kết quả Simulation trên GTKWave



- Thử nghiệm thành công trên hai kịch bản tối hạn (Critical Scenarios).

Phân tích định thời Waveform:

1. Data Forwarding Interval

Tại khoảng thời gian 10ns - 20ns:

- Cặp cổng giám sát phát hiện `ex_rs2_addr` trùng với `mem_rd_addr` (0x0C).
- `forward_b` ngay lập tức phản hồi mã 2'b10, thực hiện đi tắt dữ liệu từ tầng EX/MEM.

2. Memory Stall & Control Flush

Tại khoảng thời gian 30ns - 50ns:

- Khi `mem_wait` và `redirect_taken` đồng loạt dựng lên mức 1.
- Hệ thống dựng toàn bộ các cờ hold để đóng băng Pipeline.
- Cờ clear bị chặn về 0, bảo vệ an toàn lệnh chờ bộ nhớ khỏi bị xóa nhầm.